

CANDY DIALOGUE ENGINE

Media Commands

1.1.0

Media commands allow you to play or display videos, audio and images, and control playback precisely so they are always perfectly synchronized with your dialogues.

Media commands are split across three categories: Video, Audio and Image commands. All categories have the same commands -- just for different media formats -- except for formats where commands wouldn't be relevant (e.g. images don't have use for volume control).

Candy DE also supports animated images: you'll need to save each frame as a separate image file, not as a single file like a GIF. Candy DE will iterate through the frame files, swapping them at a customizable rate to create a visual animation.

Note that none of the media commands are intended for media that are only ever closed, stopped or otherwise controlled through dialogues. If you want media to be controlled manually (e.g. through UI buttons, by the players) you should use the §Input command and create an input UI that features media players and controls of your choice (see the Input Commands guide).

Default File Extensions

Inside the script assigned to each media player, you'll find a variable that defines the default file extension to use: `default_image_extension`, `default_audio_extension`, or `default_video_extension`.

These default extensions are used by media commands when the file reference they are provided doesn't include the extension. Since these variables are found inside the media players, each player can use a different extension if required.

Media Players

Before using any media commands, you need media players. By default, the `Candy_UI` scene has several under the "MediaPlayers" node:

VideoPlayer → For videos.

ImagePlayer → For images that don't occupy the full screen.

SplashPlayer → For full-screen images, like splash screens.

SoundPlayer → For sound effects.

AmbientPlayer → For ambient sounds.

MusicPlayer → For musics.

SpeechPlayer → For audio speech not associated to spoken lines.

These media players, are all referenced in the **media_players_locations** dictionary in **Candy_Engine.gd**. Each player has a key with its node path from the **Candy_UI** scene root:

```
#^ Media Player paths:
#TODO: Edit paths and add media player path keys as needed.
var media_players_locations = {
    "Image": "MediaPlayers/ImagePlayer",
    "Splash": "MediaPlayers/SplashPlayer",
    "Video": "MediaPlayers/VideoPlayer",
```

You can freely add any media players you want: either add them as children of the MediaPlayer node, or anywhere else you'd like in your game. Then, assign them a new key in the dictionary. If your media players are not located in the **Candy_UI** scene, just write a path from the root:

```
"Combat_SFX": "/root/game/combat/sfx_player",
```

If you create your own media players, assign them the corresponding script or they won't work with media commands:

TextureRect (or compatible image display nodes) → **Image_Player.gd**

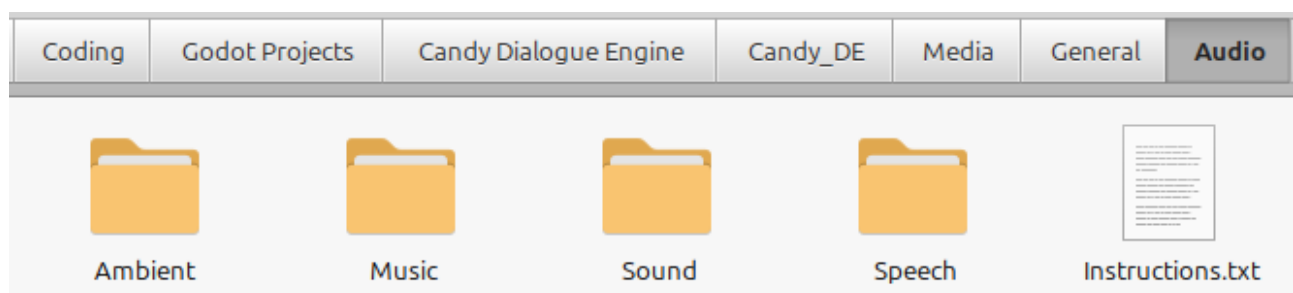
AudioStreamPlayer → **Audio_Player.gd**

VideoStreamPlayer → **Video_Player.gd**

Folders

Each media player requires its own folder for files to play. By default, these folders are found in **Candy_DE/Media/Images**, **Candy_DE/Media/Audio**, and **Candy_DE/Media/Videos**.

Inside the folder that matches their media format, media players should have their own folder, with the same name as their key in the **media_players_locations** dictionary:



For animated images, you must create a folder named "Animated" inside the folder of any image player. Then, create a subfolder for each animated image inside that "Animated" folder.

For each animated image, save its frames as individual image files, inside the image's subfolder. When you play the animated image, Candy DE will display and swap the images, selecting them in alphabetical order.

This means that you can name image frames anything you want, as long as the alphabetical order matches the order they're supposed to be displayed in: "A"/"B"/"C" or "frame_01"/"frame_02"/"frame_03", etc.

For numbers, 0-padding (adding '0' before numbers) is required:

frame_9 → Here, frame_10 will be displayed before frame_9 because 1 comes before 9.
frame_10

frame_09 → Here, frame_10 will correctly be displayed after frame_09: 0 comes before 1.
frame_10

Playing Media

Media is played with the \$Image, \$Audio and \$Video commands.

All three have the following data in common:

Image/Audio/Video Player

This is the media player you want to use. Type a key from the media_players_locations dictionary (e.g. "Music"), or provide a path to any media player anywhere in your project tree using the node_symbol as a prefix:

\$game/combat/sfx_player

Note: don't add "/root/", Candy DE already looks under /root/ by default, and don't add quotation marks.

Notice the (:) button next to the Node field? If you've provided the path to your project in the Candy Dialogue Creator settings, and if each player has its own folder in the correct location in your resources, Candy DC will immediately know which media players your project has and will list them for quick selection.

Image/Audio/Video File

This is the file you want to play. If you've assigned a media player to the Node field, the (:) button will display a list of all files for that player. Best of all: you can even preview them by hovering them in the list or in the Media field!

For animated images, provide the name of the folder that contains the image frames for the animated image you want. If you toggle the 'Animated' button to enable it, the (:) button will display the available animated image folders.

Loops

The Loop field indicates the number of times the media should play before stopping. A value of 0 causes infinite looping. Set to 1 to play the animation only once.

Wait Time & Wait Loops

You can pause dialogue processing while the media is playing:

The **Wait Time** field represents the amount of time to pause dialogue for and is provided in the **HH:MM:SS.00** format (Hours:Minutes:Seconds.decimals) which can be written multiple ways:

1:20:34 → 1 hour 20 minutes 34 seconds

03:44 → 3 minutes 44 seconds

140:05 → 140 minutes 5 seconds

0:130:90.3 → 0 hours, 130 minutes, 90.3 seconds

2.5 → 2.5 seconds

Whatever syntax you choose, Candy DE will calculate correctly as long as you use colon (:) to separate hours, minutes and seconds, and period (.) to separate decimals.

For videos, you can also indicate a specific frame. Write 'f=' or 'F=' followed by the frame number:

f=120

If the time or frame provided exceeds the length of the media, the excess will carry over to the next loops, if any. This will not cause the media to play for more loops than indicated in the Loop field.

In the **Wait Loops** field, you can indicate the number of loops to pause media for. For example, if you write '3', dialogue processing will be paused until 3 loops have played (or until all loops have played if you indicated a smaller number in the Loop field).

Time and Wait can be combined: for example, if you input '2:00' in the Time field, and '4' in the Wait field, dialogue will be paused for 4 loops and an extra 2 minutes.

To ensure dialogue pauses for a single loop, write Time = 0 and Wait = 1. To prevent pausing, write 0 for both.

Text & Portrait

The §Image and §Video commands also have the **Show/Hide Text** and **Show/Hide Portrait** fields in common: these display dialogue text and/or portraits above the media, or hide them underneath. This is done by changing their Z index.

Set to -1 to hide, 1 to show above the media, and 0 to ignore.

Static/Animated Button

The **Play Image** field indicates to Candy DE whether it should load a static image file, or an animated image made of multiple frames.

Be sure to enable 'Animated' if you want to display an animated image, or Candy DE will try displaying a static image instead.

Duration

Since images don't have a set duration like audio or video files, the §Image command has a **Duration** field, where you can type how long an image should be displayed on screen (in seconds).

If you also set the `$Image` command to loop, it will repeat the duration timer as many times as the loop value. So an image with a duration of 5 seconds and a loop value of 3 will display for $3 \times 5 = 15$ seconds.

For animated images, you can instead enter the FPS rate that Candy DE should use. FPS is the number of images it will display per second. Or in other words: the animation speed. In other words, if an Animated image is made of 15 frames, and you give it a Duration of 1.5, it will play the animation over 10 seconds.

Note that you can set a default FPS rate in the `animated_image_fps` variable in [Candy_Properties.gd](#). Candy DE will use that default rate if you leave the FPS field empty. Otherwise, if you enter a value in the FPS field, it will override the default rate.

Dialogue Wait

The `$I_Wait`, `$A_Wait` and `$V_Wait` commands can be used to pause dialogue processing until a given media has reached a specific timestamp. In the case of videos, you can also specify a specific frame.

Just like `$Image`, `$Audio` and `$Video`, you must provide a media player in the **Image/Audio/Video Player** field so the command knows which media player it's supposed to affect.

You also need to provide values for **Wait Time** and **Wait Loops**, and you have the option to **Show/Hide Text** and **Show/Hide Portrait**.

Usage

As mentioned previously, `$Image`, `$Audio` and `$Video` commands also let you pause dialogue processing. You can use `$I_Wait`, `$A_Wait` and `$V_Wait` after a media has started playing, rather than immediately at the start, or after several dialogue lines have been processed.

Media Pause

`$I_Pause`, `$A_Pause` and `$V_Pause` will pause a playing media not for a set duration, but until a specified number of dialogue lines have been processed.

Simply provide a media player in the **Image/Audio/Video Player** field, then a number of lines to pause for in the **Number Of Lines** field.

If you only want to count Spoken Lines (not commands), write '0' in the **All** field. If you want to count all lines, write '1'.

Keep in mind that `$Comment` and `$LM` (Line Marker) commands are never counted.

Alternately, you can give the **Number Of Lines** field a value of 0: this will not resume after any number of lines have been processed. Instead, media will only resume with a `$I/A/V_Resume` command.

There is no 'best' method to resume dialogue. It might be easier to set the **Number Of Lines** value to 0, and to add Media Resume commands where needed, rather than counting lines. But using a line count may also be helpful in some cases.

Like §Image and §Video, §I_Wait and §V_Wait, you have the option to **Show/Hide Text** and **Show/Hide Portrait**.

Media Resume

§I_Resume, §A_Resume and §V_Resume will resume a media that has been paused with the Media Pause commands. This will work even if the media was set to unpause after a specific number of lines have been processed.

Just provide a media player in the **Image/Audio/Video Player** field to make it resume.

Like §Image and §Video, §I_Wait and §V_Wait, you have the option to **Show/Hide Text** and **Show/Hide Portrait**.

Media Skip

§A_Skip and §V_Skip can be used to make audio or video media skip forward or back to a specific timestamp or frame. The skip command is not available for image media, whether static or animated.

In the **Image/Audio/Video Player** field, provide the media player to target. In the **Skip Time** field, provide a timestamp in the **HH:MM:SS.00** format or a video frame in the "f=" format (see Playing Media above for more details).

You can prefix the value in the **Skip Time** field with '+' or '-' to skip relatively from the current timestamp or frame. For example, if the current timestamp is 3:00 and you set Time to +1:30, it will skip to 4:30. This works with video frames as well (e.g. "+f=120").

Media Volume

§A_Volume and §V_Volume can be used to adjust the volume of an audio or video player.

As with other media commands, start by providing a media player in the **Image/Audio/Video Player** field.

Then, write the volume amount or level in the **Volume Value** field.

To increase or decrease the volume by a set amount, prefix the value with "+" or "-". For example, "+3.0" will increase the volume by 3.0, while "3.0" will set the volume to 3.0.

If you want the volume to change progressively, write a value in the **Volume Progression** field, in seconds. For example, "5.2" will make the volume change over 5.2 seconds.

If you want dialogue to pause until the volume has reached the expected value, prefix the **Volume Progression** value with "!" (for example: "!5.2"). If the "!" symbol is not provided, dialogue will continue while the volume changes.

Finally, if you just want volume to change instantly, set the **Volume Progression** value to 0.

Media Show/Hide

\$I_Hide and \$V_Hide are used to move the text display and/or the portrait display in front or behind the designated media player.

As with other media commands, start by providing a media player in the **Image/Audio/Video Player** field, assign values to **Show/Hide Text** and **Show/Hide Portrait**.

Media Stop

\$I_Stop, \$A_Stop and \$V_Stop end a playing media. Just provide a reference to a media player in the **Image/Audio/Video Player** field, as with all media commands.